

Jour 4 : DevSecOps et GitOps - Securite et Deploiement Continu

Table des matieres

1. [Programme de la journee](#1-programme-de-la-journee)
2. [Partie 1 : DevSecOps - Shift Left Security](#2-partie-1--devsecops---shift-left-security)
3. [Partie 2 : SAST, DAST et analyse de dependances](#3-partie-2--sast-dast-et-analyse-de-dependances)
4. [Partie 3 : GitOps - Principes et mise en pratique](#4-partie-3--gitops---principes-et-mise-en-pratique)
5. [Partie 4 : Deploiement continu et environnements](#5-partie-4--deploiement-continu-et-environnements)
6. [Recapitulatif et checklist](#6-recapitulatif-et-checklist)

1. Programme de la journee

Horaire	Sujet
09h00 - 10h30	DevSecOps - Integrer la securite dans le pipeline
10h30 - 10h45	Pause
10h45 - 12h15	SAST, DAST et analyse de dependances
12h15 - 13h30	Pause dejeuner
13h30 - 15h00	GitOps - Principes et mise en pratique
15h00 - 15h15	Pause
15h15 - 16h45	Deploiement continu et environnements
16h45 - 17h00	Recapitulatif et Q&A

2. Partie 1 : DevSecOps - Shift Left Security

2.1 Qu'est-ce que DevSecOps ?

Vous connaissez deja le **DevOps** : c'est l'union du developpement (Dev) et des operations (Ops) pour livrer plus vite et plus fiable. Le **DevSecOps** ajoute la **securite (Sec)** directement au coeur de ce processus.

****Analogie : la construction d'une maison****

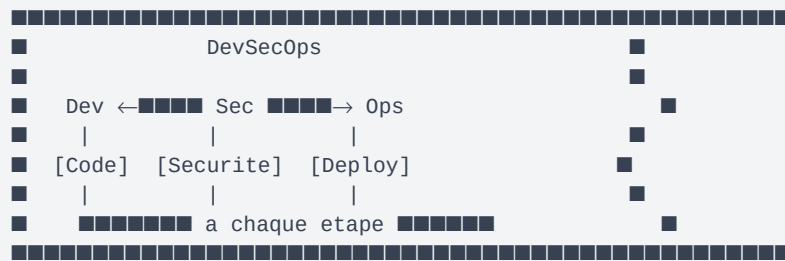
Imaginez que vous construisez une maison.

- ****Approche traditionnelle**** : vous construisez toute la maison, puis a la fin un expert securite vient verifier les serrures, les alarmes, la solidite des murs. S'il trouve un probleme, il faut parfois demolir des murs entiers pour corriger.
- ****Approche DevSecOps**** : l'expert securite est present DES LE DEBUT. Il verifie les fondations, chaque mur, chaque porte au fur et a mesure. Les problemes sont detectes et corriges immediatement, quand c'est facile et peu couteux.

Approche traditionnelle :

```
Dev ██████████> Ops ██████████> Sec (trop tard !)  
[Ecrire code]   [Deployer]       [Verifier securite]  
                                   |  
                                   "Oups, faille critique..."  
                                   "Il faut tout refaire !"
```

Approche DevSecOps :



2.2 Le concept "Shift Left"

"Shift Left" signifie litteralement **"decaler vers la gauche"**. Sur une frise chronologique du developpement, la gauche represente le debut du projet et la droite la mise en production.

****Analogie : la correction de copies****

Imaginez un professeur qui corrige des copies d'examen :

- ****Sans Shift Left**** : le professeur attend que tous les eleves aient rendu leur copie finale pour corriger. Il decouvre que la moitie de la classe n'a pas compris le sujet. Trop tard pour corriger le tir.
- ****Avec Shift Left**** : le professeur fait des mini-controles chaque semaine. Il detecte les incomprehensions des la 2eme semaine et peut reexpliquer. A l'examen final, tout le monde est pret.

[illegible]

2.3 Pourquoi le Shift Left est crucial

Le cout de correction d'une faille augmente de facon exponentielle au fil du temps :

Moment de detection	Cout de correction	Exemple
Pendant l'ecriture du code	1x (reference)	Le developpeur corrige en 5 min
Pendant le build	5x	Refactoring d'une fonction
Pendant les tests	15x	Modification de l'architecture
Pendant le staging	60x	Hotfix + re-deploiement
En production	100x	Incident, perte de donnees

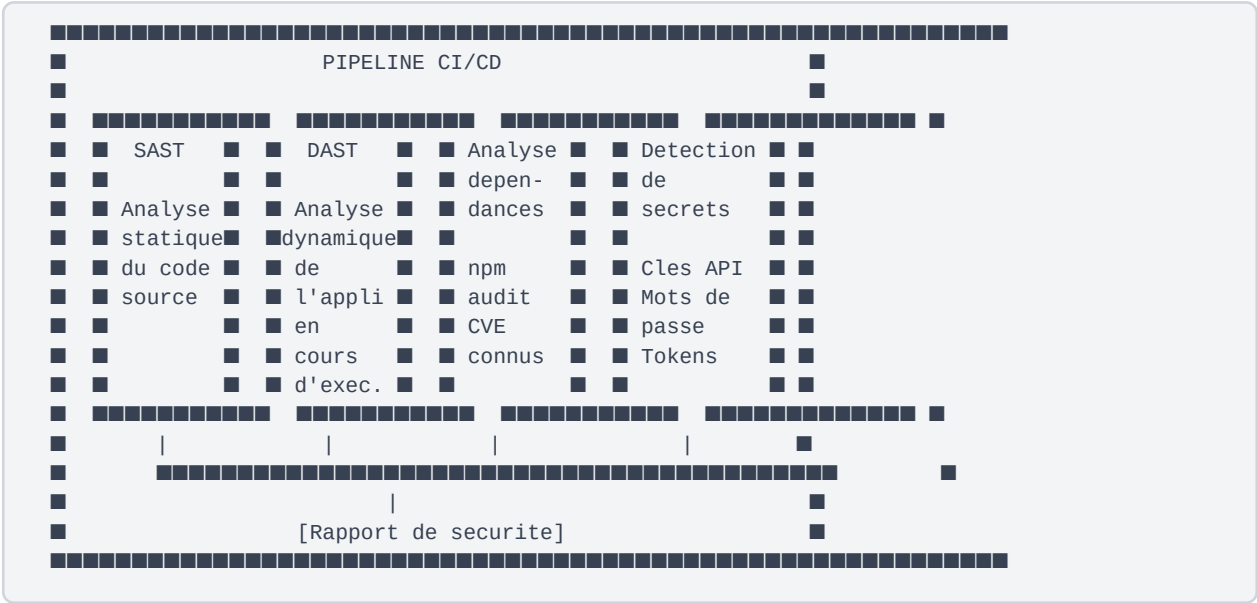
****Analogie : la faute d'orthographe dans un livre****

Corriger une faute d'orthographe coute :

- ****En ecrivant**** : vous appuyez sur la touche "retour" (gratuit)
- ****A la relecture**** : vous marquez au stylo et reecrivez (5 minutes)
- ****A l'impression**** : vous devez reimprimer des pages (couteux)
- ****Apres distribution**** : vous devez rappeler tous les livres (tres couteux)
- ****Apres que le journal l'a repere**** : atteinte a la reputation (inestimable)

2.4 Les types de tests de securite dans le CI/CD

Voici les différentes couches de sécurité que l'on peut intégrer dans un pipeline :



Type de test	Quoi ?	Quand ?	Analogie
SAST	Analyse le code source	A chaque commit	Relire une lettre avant envoi
DAST	Teste l'application en execution	Apres deploiement	Tester les serrures d'une maison
Dependances	Verifie les librairies externes	A chaque commit	Verifier que vos ingredients ne sont pas perimes
Secrets	Detecte mots de passe dans le code	A chaque commit	Verifier qu'on n'a pas laisse ses cles sur la porte

2.5 Un pipeline DevSecOps complet

Voici a quoi ressemble un pipeline avec la securite integree a chaque etape :

```
# =====
# Pipeline DevSecOps complet
# La securite est presente a CHAQUE stage
# =====

stages:
  - validate
  - build
  - test
  - security
  - deploy

# --- Stage VALIDATE : deja de la securite ---
lint-code:
  stage: validate
  script:
    - npm ci
    - npm run lint
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == "main"'

# --- Stage SECURITY : les scans dedies ---
sast-scan:
  stage: security
  script:
    - echo "Analyse SAST en cours..."
    - npm run security-check
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

dependency-scan:
  stage: security
  script:
    - npm audit --audit-level=high
  allow_failure: true
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == "main"'

secret-detection:
  stage: security
  script:
    - echo "Recherche de secrets dans le code..."
    - git log --all --diff-filter=A --name-only --pretty=format: | xargs grep -l "password\|secret\|api_key"
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
```

****Point cle**** : remarquez que le stage `security` s'execute AVANT le `deploy`.
Si une faille est detectee, le code ne sera JAMAIS deploye.

3. Partie 2 : SAST, DAST et analyse de dependances

3.1 SAST - Static Application Security Testing

Qu'est-ce que le SAST ?

Le SAST analyse votre **code source sans l'exécuter**. Il lit le code ligne par ligne et cherche des patterns connus de vulnerabilites.

****Analogie : le correcteur orthographique de Word****

Quand vous tapez du texte dans Word, le correcteur souligne les fautes en rouge SANS avoir besoin d'imprimer le document. Il analyse le texte "a froid".

Le SAST fait pareil avec votre code : il le lit et repere les erreurs de securite sans avoir besoin de lancer l'application.

Que detecte le SAST ?

Vulnerabilite	Exemple concret	Risque
Injection SQL	<code>query("SELECT * FROM users WHERE id=" + userInput)</code>	Vol de donnees
XSS (Cross-Site Scripting)	<code>innerHTML = userInput</code>	Execution de code malveillant
Hardcoded secrets	<code>const API_KEY = "sk-abc123def456"</code>	Acces non autorise
Path traversal	<code>readFile("./uploads/" + userInput)</code>	Lecture de fichiers sensibles
Mauvaise gestion d'erreurs	<code>catch(e) { /* rien */ }</code>	Problemes masques

Exemple : code vulnerable vs code corrige

```
// =====
// EXEMPLE 1 : Injection SQL
// =====

// VULNERABLE - le SAST va detecter ceci
app.get('/user', (req, res) => {
  const userId = req.query.id;
  // DANGER : l'input utilisateur est directement dans la requete SQL
  db.query("SELECT * FROM users WHERE id = " + userId);
  // Un attaquant peut envoyer : ?id=1 OR 1=1
  // Ce qui donne : SELECT * FROM users WHERE id = 1 OR 1=1
  // Resultat : TOUTES les donnees sont renvoyees !
});

// CORRIGE - utilisation de requetes parametrees
app.get('/user', (req, res) => {
  const userId = req.query.id;
  // SECURISE : le "?" est un parametre, l'input est echappe
  db.query("SELECT * FROM users WHERE id = ?", [userId]);
  // Meme si l'attaquant envoie ?id=1 OR 1=1
  // La requete traite "1 OR 1=1" comme une simple chaine de texte
});
```

```
// =====
// EXEMPLE 2 : XSS (Cross-Site Scripting)
// =====

// VULNERABLE
app.get('/search', (req, res) => {
  const searchTerm = req.query.q;
  // DANGER : l'input utilisateur est affiche tel quel dans le HTML
  res.send(`

# Resultats pour : ${searchTerm}</h1>`); // Un attaquant peut envoyer : ?q=<script>alert('pirate')</script> // Le navigateur va executer ce script ! }); // CORRIGE - echappement des caracteres speciaux HTML const escapeHtml = require('escape-html'); app.get('/search', (req, res) => { const searchTerm = escapeHtml(req.query.q); // SECURISE : les caracteres < > sont convertis en &lt; &gt; res.send(`Resultats pour : ${searchTerm}</h1>`); });


```

Configurer le SAST dans GitLab CI

GitLab fournit des **templates de securite** prêts a l'emploi :

```
# =====
# Methode 1 : utiliser le template GitLab officiel
# =====

include:
  - template: Security/SAST.gitlab-ci.yml

# C'est tout ! GitLab va automatiquement :
# - Detecter le langage de votre projet
# - Choisir le bon outil SAST
# - Executer l'analyse
# - Generer un rapport
```

```
# =====
# Methode 2 : configuration manuelle avec un outil comme Semgrep
# =====

stages:
  - security

sast-semgrep:
  stage: security
  image: returntocorp/semgrep
  script:
    - semgrep --config auto --json --output semgrep-report.json .
  artifacts:
    paths:
      - semgrep-report.json
    when: always
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == "main"'
```

Outils SAST populaires

Outil	Langages supportes	Gratuit ?
-------	--------------------	-----------

Semgrep	30+ langages	Oui (version de base)
Bandit	Python	Oui
ESLint Security	JavaScript/TypeScript	Oui
Brakeman	Ruby on Rails	Oui
SpotBugs	Java	Oui
GitLab SAST	Multi-langages (via templates)	Oui (dans GitLab)

3.2 DAST - Dynamic Application Security Testing

Qu'est-ce que le DAST ?

Le DAST teste votre application **pendant qu'elle est en cours d'exécution**.

Il envoie des requetes malveillantes et observe comment l'application reagit.

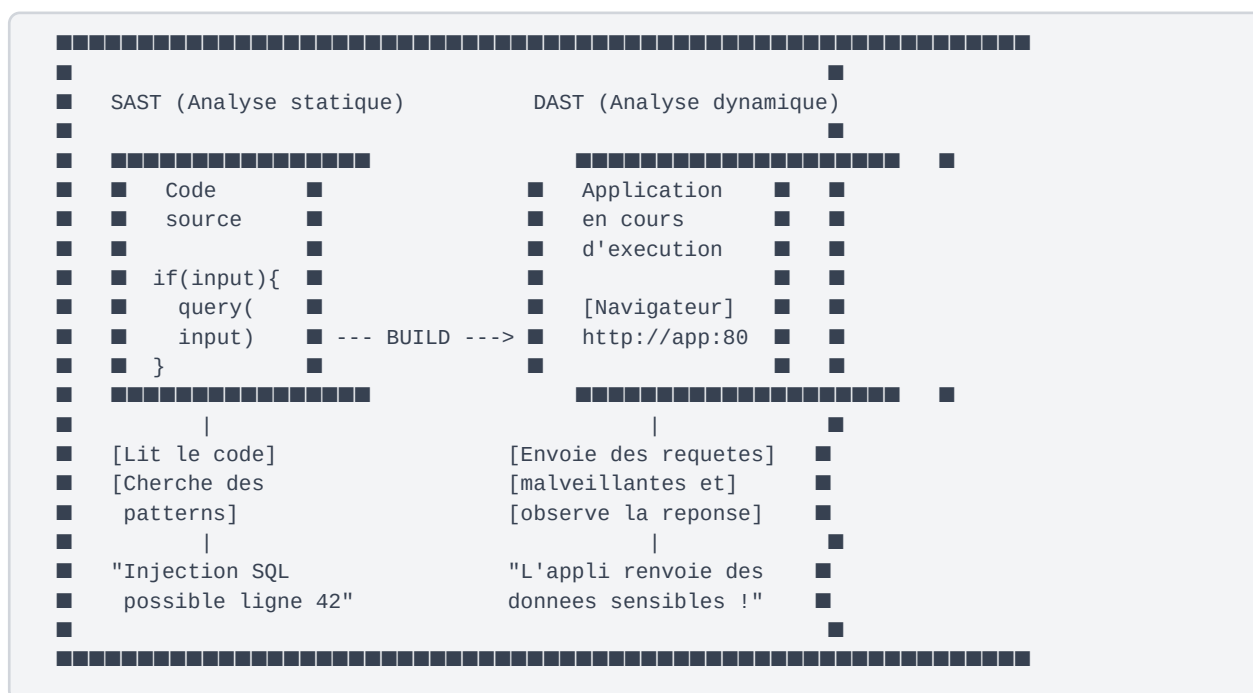
****Analogie : le test d'intrusion d'un batiment****

Le SAST, c'est lire les plans du batiment pour trouver les faiblesses.

Le DAST, c'est engager quelqu'un pour ESSAYER d'entrer par effraction.

- Le SAST regarde les plans : "Cette porte n'a pas de serrure prevue."
- Le DAST teste physiquement : "Je pousse cette porte... et elle s'ouvre !"

Les deux sont complementaires : le SAST trouve des failles theoriques, le DAST trouve des failles reelles.



Que detecte le DAST ?

Vulnerabilite	Comment le DAST la trouve
Injection SQL	Envoie ' OR 1=1 -- dans les champs de formulaire
XSS	Envoie <script>alert(1)</script> et verifie si ca s'execute

CSRF	Verifie l'absence de tokens anti-CSRF
Headers manquants	Verifie que les headers de securite sont presents
Cookies non securises	Verifie les flags HttpOnly, Secure, SameSite
Exposition d'informations	Cherche des pages d'erreur qui revelent des details

Configurer le DAST dans GitLab CI

```
# =====
# DAST avec GitLab template officiel
# =====

include:
  - template: Security/DAST.gitlab-ci.yml

variables:
  DAST_WEBSITE: 'https://mon-app-staging.example.com'

# =====
# DAST avec OWASP ZAP (outil open source)
# =====

stages:
  - deploy-staging
  - dast

deploy-to-staging:
  stage: deploy-staging
  script:
    - echo "Deploiement sur staging..."
    - docker compose up -d
  environment:
    name: staging
    url: 'http://staging.example.com'

dast-zap-scan:
  stage: dast
  image: ghcr.io/zaproxy/zaproxy:stable
  script:
    - mkdir -p /zap/wrk
    - zap-baseline.py -t http://staging.example.com -r zap-report.html || true
  artifacts:
    paths:
      - zap-report.html
    when: always
  needs:
    - deploy-to-staging
```

****Point important**** : le DAST a besoin que l'application soit DEJA deployee pour pouvoir la tester. C'est pourquoi il s'execute apres le stage de deploiement staging. On ne fait JAMAIS de DAST directement en production !

SAST vs DAST : tableau comparatif

Critere	SAST	DAST
Quand ?	Avant le build	Apres le deploiement staging
Quoi ?	Analyse le code source	Teste l'application en execution
Besoin de l'appli ?	Non	Oui, elle doit tourner


```
# =====
# Script avance : bloquer le pipeline si vulnerabilite critique
# =====

dependency-audit-strict:
  stage: security
  image: node:18-alpine
  script:
    - npm ci
    - echo "Lancement de l'audit de securite..."
    - npm audit --json > audit-results.json || true
    - |
      CRITICAL=$(cat audit-results.json | grep -c '"severity":"critical"' || echo "0")
      HIGH=$(cat audit-results.json | grep -c '"severity":"high"' || echo "0")
      echo "Vulnerabilites critiques : $CRITICAL"
      echo "Vulnerabilites hautes : $HIGH"
      if [ "$CRITICAL" -gt "0" ]; then
        echo "BLOQUE : des vulnerabilites critiques ont ete trouvees !"
        exit 1
      fi
  artifacts:
    paths:
      - audit-results.json
    when: always
```

3.4 Detection de secrets

Le probleme des secrets commites par accident

Un **secret**, c'est toute information sensible qui ne devrait JAMAIS se retrouver dans votre code source :

- Cles d'API (AWS, Google Cloud, Stripe...)
- Mots de passe de base de donnees
- Tokens d'authentification (JWT, OAuth)
- Cles privees SSH
- Certificats SSL

****Analogie : ecrire son code de carte bancaire sur un Post-it****

Imaginez que vous ecrivez votre code de carte bancaire sur un Post-it et que vous le collez sur le tableau d'affichage de votre bureau. Tout le monde peut le voir. C'est exactement ce qui se passe quand vous mettez un mot de passe dans votre code et que vous le poussez sur un depot Git public (ou meme prive, car d'autres collegues y ont acces).

Et le pire : meme si vous supprimez le Post-it ensuite, tout le monde l'a deja vu. En Git, meme si vous supprimez le secret dans un nouveau commit, il reste dans l'HISTORIQUE Git pour toujours.

■ CE QU'IL NE FAUT JAMAIS FAIRE :

```
// config.js
const DB_PASSWORD = "SuperSecret123!";           // NON !
const API_KEY = "sk-abc123def456ghi789";         // NON !
const AWS_SECRET = "wJalrXUtnFEMI/K7MDENG/bP";   // NON !
```

■ CE QU'IL FAUT FAIRE :

```
// config.js
const DB_PASSWORD = process.env.DB_PASSWORD;     // OUI !
const API_KEY = process.env.API_KEY;             // OUI !
const AWS_SECRET = process.env.AWS_SECRET;       // OUI !

// Les valeurs reelles sont stockees dans :
// - Les variables CI/CD de GitLab (Settings > CI/CD > Variables)
// - Un gestionnaire de secrets (Vault, AWS Secrets Manager)
// - Un fichier .env qui est dans le .gitignore
```

Configurer la detection de secrets

```
# =====
# Detection de secrets dans GitLab CI
# =====

# --- Methode 1 : template GitLab officiel ---
include:
  - template: Security/Secret-Detection.gitlab-ci.yml

# --- Methode 2 : avec gitleaks (outil open source) ---
secret-detection:
  stage: security
  image:
    name: zricethezav/gitleaks
    entrypoint: ['']
  script:
    - gitleaks detect --source . --report-format json --report-path gitleaks-report.json
  artifacts:
    paths:
      - gitleaks-report.json
    when: always
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == "main"'
```

Le fichier .gitignore : votre premiere ligne de defense

```
# =====
# .gitignore - fichiers a NE JAMAIS commiter
# =====

# Fichiers d'environnement (contiennent des secrets)
.env
.env.local
.env.production
.env.*.local

# Cles privees
*.pem
*.key
id_rsa
id_ed25519

# Fichiers de configuration avec des secrets
config/secrets.yml
config/credentials.yml

# Repertoire de dependances (regeneré par npm install)
node_modules/

# Fichiers de build
dist/
build/
```

3.5 Les templates de securite GitLab

GitLab fournit un ensemble de templates prêts a l'emploi pour la securite.

Voici comment les utiliser tous ensemble :

```
# =====
# Pipeline DevSecOps complet avec tous les templates GitLab
# =====

include:
  # Analyse statique du code (SAST)
  - template: Security/SAST.gitlab-ci.yml
  # Analyse des dependances
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  # Detection de secrets
  - template: Security/Secret-Detection.gitlab-ci.yml
  # Analyse des conteneurs Docker
  - template: Security/Container-Scanning.gitlab-ci.yml
  # Analyse des licences
  - template: Security/License-Scanning.gitlab-ci.yml
  # Analyse dynamique (DAST)
  - template: Security/DAST.gitlab-ci.yml

stages:
  - build
  - test
  - deploy-staging
  - dast
  - deploy-production

variables:
  DAST_WEBSITE: 'https://staging.example.com'
```

****Avantage des templates GitLab**** : ils generent automatiquement des rapports visibles dans l'onglet "Security" de votre merge request. Vous pouvez voir toutes les vulnerabilites detectees AVANT de fusionner le code.

[illegible]

4. Partie 3 : GitOps - Principes et mise en pratique

4.1 Qu'est-ce que le GitOps ?

Le **GitOps** est une methode de gestion de l'infrastructure et des deploiements ou **Git est la seule source de verite**. Tout ce qui est deploye est decrit dans des fichiers stockes dans Git.

****Analogie : le plan d'architecte****

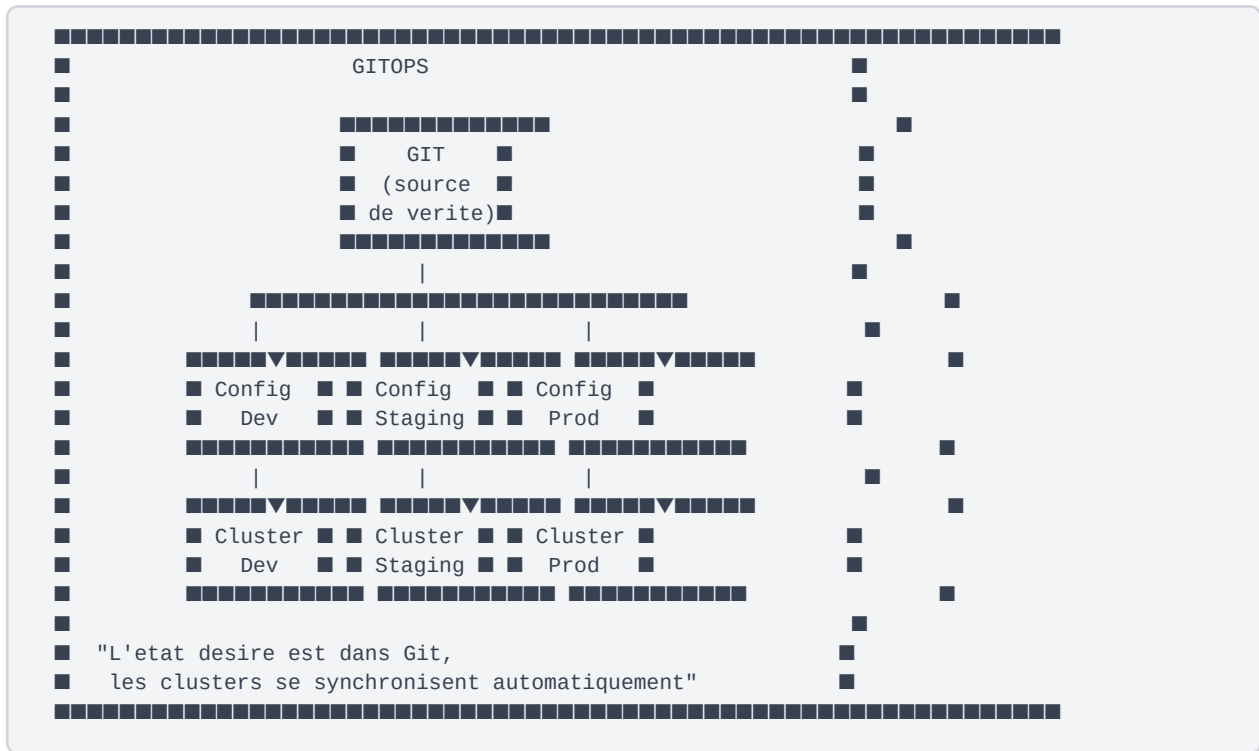
Imaginez la construction d'un immeuble :

- ****Sans GitOps**** : le chef de chantier donne des instructions a la voix.
Chaque ouvrier fait ce qu'il a compris. Si un ouvrier est absent le lendemain, personne ne sait exactement ce qu'il a fait. Il n'y a pas de trace écrite.
Si l'immeuble a un probleme, personne ne sait retrouver ce qui a change.
- ****Avec GitOps**** : tout est écrit dans un plan d'architecte (Git).
Chaque modification du plan est enregistrée avec qui l'a faite et pourquoi.
Si on veut revenir en arriere, on reprend le plan precedent.
Tout le monde regarde le meme plan.

Imaginez la construction d'un immeuble :

- **Sans GitOps** : le chef de chantier donne des instructions a la voix.
Chaque ouvrier fait ce qu'il a compris. Si un ouvrier est absent le lendemain, personne ne sait exactement ce qu'il a fait. Il n'y a pas de trace ecrite.
Si l'immeuble a un probleme, personne ne sait retrouver ce qui a change.
- **Avec GitOps** : tout est ecrit dans un plan d'architecte (Git).
Chaque modification du plan est enregistree avec qui l'a faite et pourquoi.
Si on veut revenir en arriere, on reprend le plan precedent.
Tout le monde regarde le meme plan.

- Imaginez la construction d'un immeuble :
- **Sans GitOps** : le chef de chantier donne des instructions a la voix.
Chaque ouvrier fait ce qu'il a compris. Si un ouvrier est absent le lendemain, personne ne sait exactement ce qu'il a fait. Il n'y a pas de trace ecrite.
Si l'immeuble a un probleme, personne ne sait retrouver ce qui a change.
 - **Avec GitOps** : tout est ecrit dans un plan d'architecte (Git).
Chaque modification du plan est enregistree avec qui l'a faite et pourquoi.
Si on veut revenir en arriere, on reprend le plan precedent.
Tout le monde regarde le meme plan.



4.2 Les 4 principes du GitOps

Principe 1 : Configuration declarative

On decrit l'**etat desire** du systeme, pas les etapes pour y arriver.

****Analogie : commander au restaurant****

- ****Imperatif**** (non GitOps) : "Prenez de la farine, ajoutez de l'eau, petrissez, faites cuire a 200 degres pendant 30 minutes, ajoutez de la sauce tomate..."
- ****Declaratif**** (GitOps) : "Je veux une pizza margherita."

En declaratif, on dit **CE QU'ON VEUT**, pas **COMMENT LE FAIRE**.
Le systeme se debrouille pour atteindre l'etat desire.

```
# =====
# Exemple IMPERATIF (pas GitOps) :
# On dit au systeme QUOI FAIRE etape par etape
# =====

# deploy.sh
# docker pull mon-app:v2.0
# docker stop mon-app-container
# docker rm mon-app-container
# docker run -d --name mon-app-container -p 80:3000 mon-app:v2.0
# Si une etape echoue, le systeme est dans un etat inconnu !
```


- Quelqu'un propose un changement
- Un ou plusieurs collègues relisent (review)
- Si approuve, le changement est fusionné
- Le déploiement se fait automatiquement



Principe 4 : Reconciliation automatique

Un **agent GitOps** surveille en permanence le depot Git ET le cluster.

Si l'etat reel du cluster diverge de l'etat desire dans Git, l'agent corrige automatiquement.

****Analogie : le thermostat****

Vous reglez votre thermostat sur 21 degres (etat desire).

Si la temperature descend a 19 degres (derive), le chauffage se rallume automatiquement pour revenir a 21 degres.

Vous n'avez rien a faire : le systeme se corrige tout seul.

GitOps fait pareil : si quelqu'un modifie manuellement le cluster (par exemple, change le nombre de replicas), l'agent GitOps detecte la divergence et remet la configuration telle qu'elle est dans Git.

Boucle de reconciliation GitOps :



4.3 Pull vs Push : les deux modes de deployment

C'est une distinction fondamentale en GitOps :

Modele PUSH (traditionnel)

Le pipeline CI/CD **pousse** les changements vers le cluster.



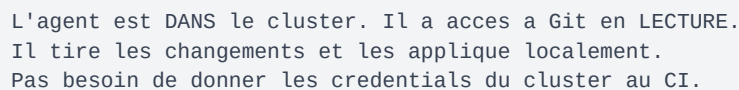
Le pipeline a les credentials pour acceder au cluster.
C'est LUI qui pousse les changements.

Inconvenients du modele Push :

- Le pipeline CI a besoin des **credentials** du cluster (risque de securite)
- Pas de reconciliation automatique (si quelqu'un modifie le cluster a la main, le pipeline ne le sait pas)
- L'etat reel du cluster peut diverger de ce qui est dans Git

Modele PULL (GitOps)

Un agent **a l'interieur du cluster** tire les changements depuis Git.



- Le pipeline CI n'a **pas besoin d'accéder au cluster** (plus sécurisé)
- Reconciliation automatique et continue
- L'état du cluster est toujours synchronisé avec Git



```
# =====
# Exemple de configuration ArgoCD
# Fichier : argocd-application.yaml
# =====

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: mon-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: 'https://gitlab.com/team/infra.git'
    targetRevision: main
    path: kubernetes/overlays/production
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: production
  syncPolicy:
    automated:
      # Synchronisation automatique
      prune: true      # Supprime les ressources obsolètes
      selfHeal: true   # Corrige les dérives automatiquement
    syncOptions:
      - CreateNamespace=true
```

Flux (FluxCD)

Flux est l'autre outil GitOps majeur, également pour Kubernetes.

Il est plus "léger" qu'ArgoCD (pas d'interface graphique par défaut).

```
# =====
# Exemple de configuration Flux
# =====

# GitRepository : où trouver la configuration
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: mon-infra
  namespace: flux-system
spec:
  interval: 1m          # Vérifier Git toutes les minutes
  url: 'https://gitlab.com/team/infra.git'
  ref:
    branch: main

---
# Kustomization : quoi déployer
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: mon-app
  namespace: flux-system
spec:
  interval: 5m
  path: ./kubernetes/overlays/production
  prune: true
  sourceRef:
    kind: GitRepository
    name: mon-infra
```

ArgoCD vs Flux : comparaison

Critere	ArgoCD	Flux
Interface	Interface web riche	CLI (Weave GitOps en option)
Complexeite	Plus lourd a installer	Plus leger
Multi-cluster	Oui, nativement	Oui, via configuration
Communaute	Tres large	Large (CNCF)
Courbe d'apprentissage	Moderee	Plus douce
Ideal pour	Grandes equipes, visibilite	Equipes DevOps, simplicité

5. Partie 4 : Deploiement continu et environnements

5.1 Continuous Delivery vs Continuous Deployment

Ces deux termes sont souvent confondus. Voici la difference :

****Analogie : la chaine de montage automobile****

- ****Continuous Delivery**** (Livraison Continue) : la voiture est assemblee, testee et prete a etre livree. Mais un humain doit appuyer sur le bouton "Envoyer au concessionnaire" pour qu'elle parte.
- ****Continuous Deployment**** (Deploiement Continu) : la voiture est assemblee, testee et envoyee AUTOMATIQUEMENT au concessionnaire sans intervention humaine.

Continuous DELIVERY :

Code ■■> Build ■■> Test ■■> Staging ■■> [BOUTON MANUEL] ■■> Production

▲
|
Un humain decide
quand deployer

Continuous DEPLOYMENT :

Code ■■> Build ■■> Test ■■> Staging ■■> Production

▲
|
AUTOMATIQUE !
Pas d'intervention
humaine

Critere	Continuous Delivery	Continuous Deployment
Deploiement en prod	Manuel (bouton a cliquer)	Automatique
Confiance requise	Moderee	Tres haute
Tests necessaires	Bons tests	Tests excellents et complets
Vitesse de livraison	Rapide (quand on decide)	Tres rapide (a chaque commit)
Adapte pour	La plupart des equipes	Equipes tres matures
Risque	Modere (validation humaine)	Faible si tests excellents

```
# =====
# Exemple Continuous DELIVERY dans GitLab CI
# Le deploiement en production necessite un clic manuel
# =====

stages:
  - build
  - test
  - deploy-staging
  - deploy-production

build-app:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/

test-app:
  stage: test
  script:
    - npm ci
    - npm test

deploy-staging:
  stage: deploy-staging
  script:
    - echo "Deploiement sur staging..."
  environment:
    name: staging
    url: 'https://staging.example.com'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# LE MOT CLE "when: manual" = bouton a cliquer dans GitLab
deploy-production:
  stage: deploy-production
  script:
    - echo "Deploiement en production..."
  environment:
    name: production
    url: 'https://www.example.com'
  when: manual
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

```
# =====
# Exemple Continuous DEPLOYMENT dans GitLab CI
# Le deploiement en production est AUTOMATIQUE
# =====

stages:
  - build
  - test
  - deploy-staging
  - integration-test
  - deploy-production

build-app:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/

test-app:
  stage: test
  script:
    - npm ci
    - npm test
    - npm run test:coverage

deploy-staging:
  stage: deploy-staging
  script:
    - echo "Deploiement automatique sur staging..."
  environment:
    name: staging
    url: 'https://staging.example.com'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# Tests supplementaires sur l'environnement staging
integration-tests:
  stage: integration-test
  script:
    - echo "Tests d'integration sur staging..."
    - npm run test:e2e -- --url https://staging.example.com
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# PAS de "when: manual" = deploiement AUTOMATIQUE
deploy-production:
  stage: deploy-production
  script:
    - echo "Deploiement automatique en production..."
  environment:
    name: production
    url: 'https://www.example.com'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

5.2 Promotion d'environnements (Dev -> Staging -> Production)

Le concept de promotion

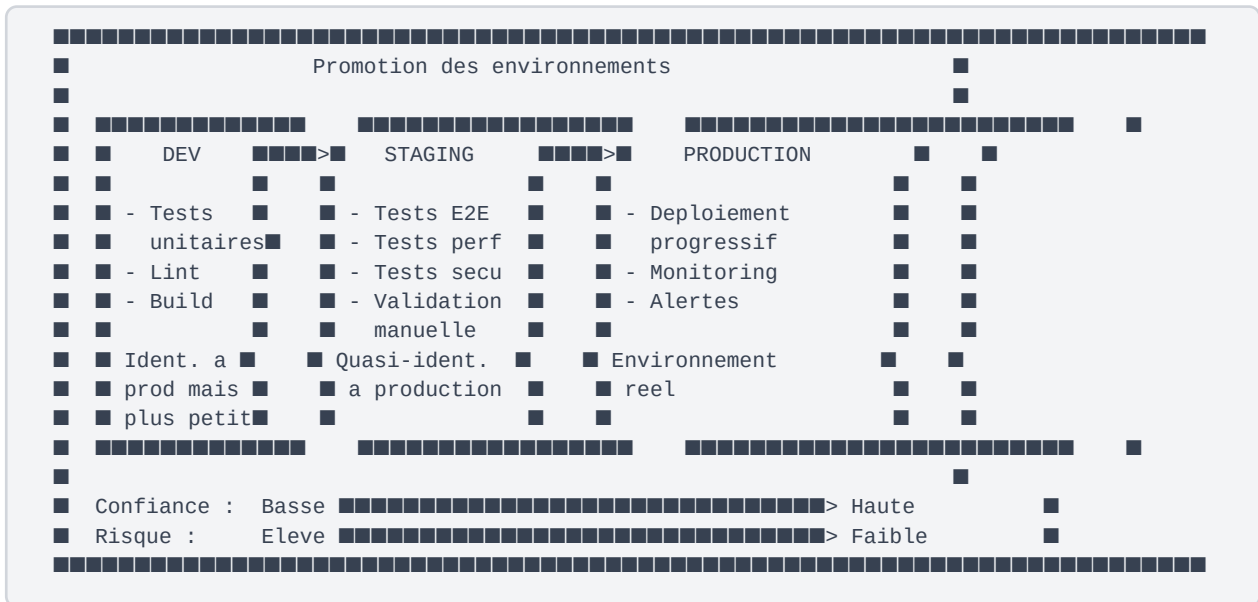
La **promotion** est le processus par lequel votre application passe d'un environnement a un autre, avec des verifications a chaque etape.

****Analogie : l'examen de passage****

Comme a l'ecole, votre application doit "passer des examens" pour avancer :

- ****Dev**** : l'eleve fait ses devoirs (le developpeur teste en local)
- ****Staging**** : l'eleve passe un examen blanc (tests sur un env. de pre-production)
- ****Production**** : l'eleve passe l'examen final (deploiement pour les vrais utilisateurs)

On ne peut pas passer a l'etape suivante sans avoir reussi la precedente.



Configuration des environnements dans GitLab CI

```
# =====
# Promotion d'environnements avec GitLab CI
# =====

stages:
  - build
  - test
  - deploy-dev
  - deploy-staging
  - deploy-production

variables:
  APP_NAME: 'mon-application'

# --- Deploiement DEV : automatique sur chaque branche ---
deploy-dev:
  stage: deploy-dev
  script:
    - echo "Deploiement sur dev pour la branche $CI_COMMIT_BRANCH"
  environment:
    name: 'dev/$CI_COMMIT_BRANCH'
    url: 'https://$CI_COMMIT_BRANCH.dev.example.com'
    on_stop: stop-dev
  rules:
    - if: '$CI_COMMIT_BRANCH != "main"'

stop-dev:
  stage: deploy-dev
  script:
    - echo "Suppression de l'environnement dev/$CI_COMMIT_BRANCH"
  environment:
    name: 'dev/$CI_COMMIT_BRANCH'
    action: stop
  when: manual
  rules:
    - if: '$CI_COMMIT_BRANCH != "main"'

# --- Deploiement STAGING : automatique sur main ---
deploy-staging:
  stage: deploy-staging
  script:
    - echo "Deploiement sur staging..."
  environment:
    name: staging
    url: 'https://staging.example.com'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# --- Deploiement PRODUCTION : manuel, uniquement depuis main ---
deploy-production:
  stage: deploy-production
  script:
    - echo "Deploiement en production..."
  environment:
    name: production
    url: 'https://www.example.com'
  when: manual
  rules:
```

5.3 Feature Flags (drapeaux de fonctionnalités)

Qu'est-ce qu'un feature flag ?

Un **feature flag** est un interrupteur dans votre code qui permet d'activer ou

****Analogie : l'interrupteur de lumiere****

Imaginez que chaque nouvelle fonctionnalité de votre application est une lampe dans une pièce. Le feature flag est l'interrupteur.

- Interrupteur ON : la fonctionnalité est visible par les utilisateurs
- Interrupteur OFF : la fonctionnalité est dans le code mais invisible

Le code est déjà déployé, mais la fonctionnalité ne s'active que quand vous "allumez" l'interrupteur. Si un problème est détecté, vous "éteignez" instantanément, sans redéployer.

[illegible]

```
Deployer v2 (flag OFF) ██████> Activator flag ██████> Bug ! ██████> Desactivator flag
```

 5 min	 instant	 10 min	 instant
Total : ~0 min de downtime			

```
// =====
// Exemple simple de feature flag dans une application Node.js
// =====

// config/feature-flags.js
const featureFlags = {
  NEW_CHECKOUT_PAGE: process.env.FF_NEW_CHECKOUT === 'true',
  DARK_MODE: process.env.FF_DARK_MODE === 'true',
  AI_RECOMMENDATIONS: process.env.FF_AI_RECO === 'true',
};

module.exports = featureFlags;

// routes/checkout.js
const flags = require('../config/feature-flags');

app.get('/checkout', (req, res) => {
  if (flags.NEW_CHECKOUT_PAGE) {
    // Nouvelle page de paiement (en cours de test)
    res.render('checkout-v2');
  } else {
    // Ancienne page de paiement (stable)
    res.render('checkout-v1');
  }
});
```

```
# =====
# Feature flags via les variables d'environnement GitLab CI
# =====

deploy-staging:
  stage: deploy-staging
  variables:
    FF_NEW_CHECKOUT: 'true'      # Active en staging pour tester
    FF_DARK_MODE: 'true'        # Active en staging pour tester
    FF_AI_RECO: 'false'         # Pas encore pret
  script:
    - echo "Deploiement staging avec feature flags..."
  environment:
    name: staging

deploy-production:
  stage: deploy-production
  variables:
    FF_NEW_CHECKOUT: 'false'     # Pas encore active en production
    FF_DARK_MODE: 'true'        # Deja valide, active en production
    FF_AI_RECO: 'false'         # Pas encore pret
  script:
    - echo "Deploiement production avec feature flags..."
  environment:
    name: production
```

Outils de feature flags

Outil	Type	Gratuit ?
LaunchDarkly	SaaS	Non (payant)
Unleash	Open source / SaaS	Oui (version OSS)
Flip	Open source	Oui
GitLab Feature Flags	Integre GitLab	Oui (dans GitLab)
Variables d'env	DIY	Oui

5.4 Canary Deployments (deploiements canari)

Qu'est-ce qu'un deploiement canari ?

Un **canary deployment** consiste a deployer la nouvelle version de l'application sur un **petit pourcentage de serveurs** d'abord, puis a augmenter progressivement.

****Analogie : le canari dans la mine****

Au 19eme siecle, les mineurs emmenaient un canari dans les mines de charbon. Si le canari tombait malade, c'etait le signe qu'il y avait du gaz dangereux et les mineurs evacuaient AVANT d'etre touches eux-memes.

En deploiement, le "canari" est un petit groupe de serveurs avec la nouvelle version. Si ce petit groupe a des problemes, on arrete le deploiement AVANT que tous les utilisateurs soient touches.

Etape 1 : Deploiement canari (5% du trafic)

Utilisateurs ██████████ 95% ████████> v1.0 (stable)

```

|
■■■ 5% ■■> v2.0 (canari)    <-- On surveille

```

Etape 2 : Si tout va bien, on augmente (25%)

Utilisateurs ██████████ 75% ████████> v1.0 (stable)

```

■■■ 25% ■> v2.0 (canari)    <-- Toujours OK

```

Etape 3 : On continue (50%)

Utilisateurs ██████████ 50% ████████> v1.0 (stable)

```

|
■■■■ 50% ■> v2.0 (canari)    <-- Ca roule !

```

Etape 4 : Deploiement complet (100%)

```
Utilisateurs ██████████ 100% ████████> v2.0 (nouvelle version)
```

Succes !

OU, si un probleme est detecte a l'etape 2 :

Utilisateurs ██████████ 100% ████████> v1.0 (rollback)

v2.0 est retire

Configuration dans GitLab CI

```
# =====
# Deploiement Canari avec GitLab CI
# =====

stages:
  - build
  - test
  - canary
  - production

deploy-canary:
  stage: canary
  script:
    - echo "Deploiement canari (5% du trafic)..."
    - echo "Surveillance pendant 10 minutes..."
    # En vrai, on utiliserait Kubernetes avec Istio ou Nginx
    # pour diriger 5% du trafic vers la nouvelle version
  environment:
    name: production
    url: 'https://www.example.com'
  when: manual
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

promote-canary:
  stage: production
  script:
    - echo "Le canari est sain, deploiement complet..."
  environment:
    name: production
    url: 'https://www.example.com'
  when: manual
  needs:
    - deploy-canary
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

rollback-canary:
  stage: production
  script:
    - echo "Probleme detecte, rollback du canari..."
  environment:
    name: production
    url: 'https://www.example.com'
    action: stop
  when: manual
  needs:
    - deploy-canary
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

5.5 Strategies de rollback

Qu'est-ce qu'un rollback ?

Un **rollback** est le retour a une version precedente de l'application quand la nouvelle version pose probleme.

****Analogie : la fonction "Annuler" (Ctrl+Z)****

Quand vous faites une erreur dans Word, vous appuyez sur Ctrl+Z pour revenir en arriere. Le rollback, c'est le Ctrl+Z du deploiement.
Plus vous pouvez "annuler" rapidement, moins les degats sont importants.

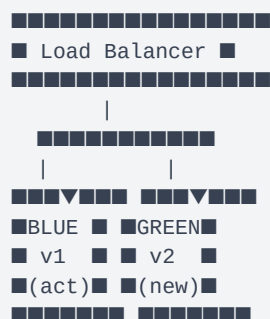
Les differentes strategies de rollback

Strategie 1 : REDEPLOYER L'ANCIENNE VERSION

v1 ■■> v2 (bug !) ■■> re-deployer v1

- + Simple a comprendre
- Prend du temps (rebuild + redeploy)
- Temps d'indisponibilite possible

Strategie 2 : BLUE-GREEN DEPLOYMENT



Si v2 a un probleme : on bascule le load balancer vers BLUE (v1)
Rollback instantane ! Les deux versions coexistent.

- + Rollback instantane (switch du load balancer)
- + Zero downtime
- Double de ressources necessaire (deux environnements complets)

Strategie 3 : ROLLING UPDATE

Serveurs : [v1] [v1] [v1] [v1] [v1]
Etape 1 : [v2] [v1] [v1] [v1] [v1] ← 1 serveur mis a jour
Etape 2 : [v2] [v2] [v1] [v1] [v1] ← 2 serveurs mis a jour
Bug ! : [v1] [v1] [v1] [v1] [v1] ← Rollback progressif

- + Pas de double de ressources
- + Rollback possible a chaque etape
- Plus lent que blue-green

Exemple de rollback avec GitLab CI

```
# =====
# Rollback dans GitLab CI
# =====

stages:
  - build
  - deploy
  - rollback

deploy-production:
  stage: deploy
  script:
    - echo "Deploiement de la version $CI_COMMIT_SHORT_SHA"
    # Sauvegarder la version actuelle pour le rollback
    - echo "$CI_COMMIT_SHORT_SHA" > /tmp/current-version.txt
  environment:
    name: production
    url: 'https://www.example.com'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# Job de rollback : toujours manuel
rollback-production:
  stage: rollback
  script:
    - echo "Rollback vers la version precedente..."
    # En vrai, on utiliserait une commande comme :
    # kubectl rollout undo deployment/mon-app
    # ou
    # docker service update --rollback mon-app
  environment:
    name: production
    url: 'https://www.example.com'
  when: manual
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

5.6 Monitoring et observabilite dans le CI/CD

Pourquoi le monitoring est essentiel

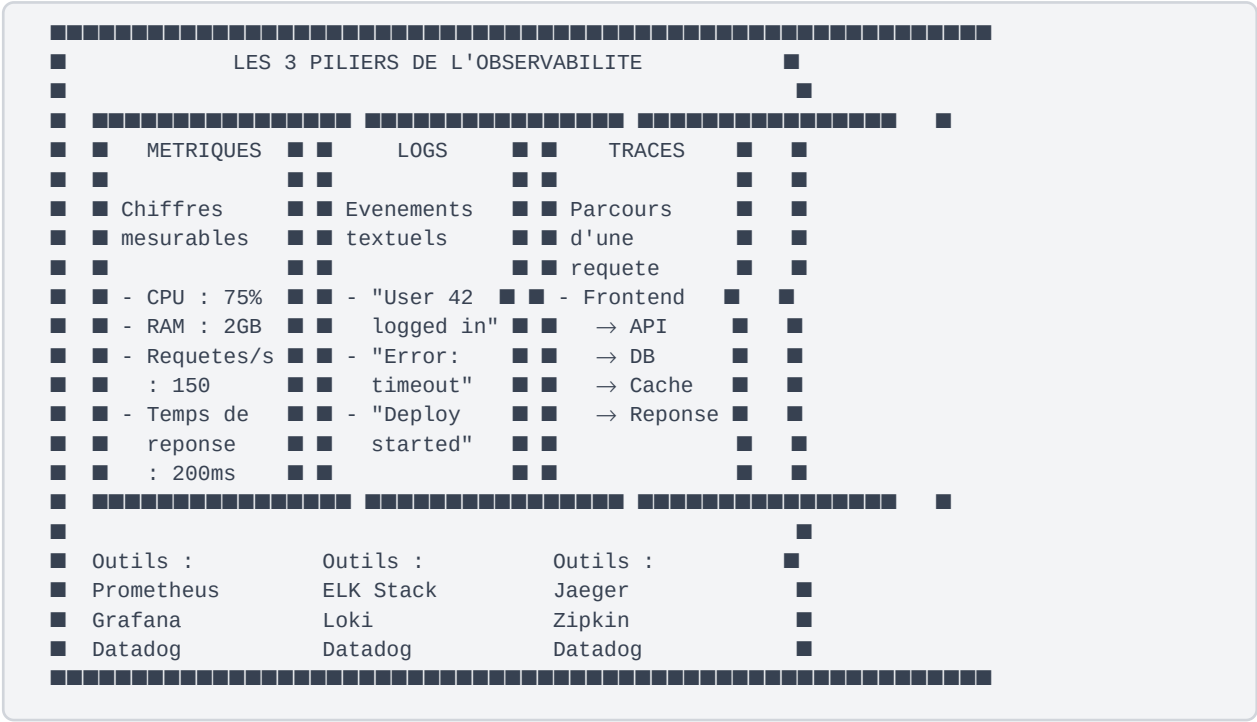
Deployer n'est que la moitié du travail. Il faut ensuite **surveiller** que tout fonctionne correctement.

****Analogie : le tableau de bord d'une voiture****

Quand vous conduisez, vous ne regardez pas seulement la route. Vous vérifiez aussi le tableau de bord : vitesse, niveau d'essence, température du moteur, voyants d'alerte...

Le monitoring, c'est le tableau de bord de votre application. Il vous dit si tout va bien ou si quelque chose commence à mal tourner.

Les trois piliers de l'observabilite



Pilier	Quoi ?	Exemple	Outil populaire
Metriques	Des chiffres dans le temps	CPU a 95% depuis 5 min	Prometheus + Grafana
Logs	Des messages textuels	"Erreur connexion base de donnees"	ELK Stack, Loki
Traces	Le chemin d'une requete	Requete: frontend -> api -> db	Jaeger, Zipkin

Integrer le monitoring dans le pipeline CI/CD

```

# =====
# Pipeline avec verification post-deploiement
# =====

stages:
  - build
  - test
  - deploy
  - verify

deploy-production:
  stage: deploy
  script:
    - echo "Deploiement en production..."
  environment:
    name: production
    url: 'https://www.example.com'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# Verification apres deploiement : le monitoring automatique
smoke-test:
  stage: verify
  script:
    - echo "=== Smoke tests post-deploiement ==="
    # Verifier que l'application repond
    - 'curl -f https://www.example.com/health || exit 1'
    # Verifier le temps de reponse
    - |
      RESPONSE_TIME=$(curl -s -o /dev/null -w "%{time_total}" https://www.example.com)
      echo "Temps de reponse : ${RESPONSE_TIME}s"
      if (( $(echo "$RESPONSE_TIME > 2.0" | bc -l) )); then
        echo "ALERTE : temps de reponse trop long !"
        exit 1
      fi
    # Verifier le code de statut
    - |
      STATUS=$(curl -s -o /dev/null -w "%{http_code}" https://www.example.com)
      echo "Code HTTP : $STATUS"
      if [ "$STATUS" != "200" ]; then
        echo "ALERTE : l'application ne repond pas correctement !"
        exit 1
      fi
  needs:
    - deploy-production
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# Verification des metriques via l'API de monitoring
check-metrics:
  stage: verify
  script:
    - echo "=== Verification des metriques ==="
    # Exemple avec Prometheus
    - |
      ERROR_RATE=$(curl -s 'http://prometheus:9090/api/v1/query?query=rate(http_errors_total[5m])' | jq '.data[0].value')
      echo "Taux d'erreur : $ERROR_RATE"
      if (( $(echo "$ERROR_RATE > 0.05" | bc -l) )); then

```

Le pipeline complet avec monitoring et rollback automatique

```
# =====
# Pipeline ideal : deploy + monitor + auto-rollback
# =====

stages:
  - build
  - test
  - security
  - deploy-staging
  - integration-test
  - deploy-production
  - verify
  - rollback

# [...stages precedents...]

deploy-production:
  stage: deploy-production
  script:
    - echo "Deploiement en production..."
  environment:
    name: production
    url: 'https://www.example.com'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

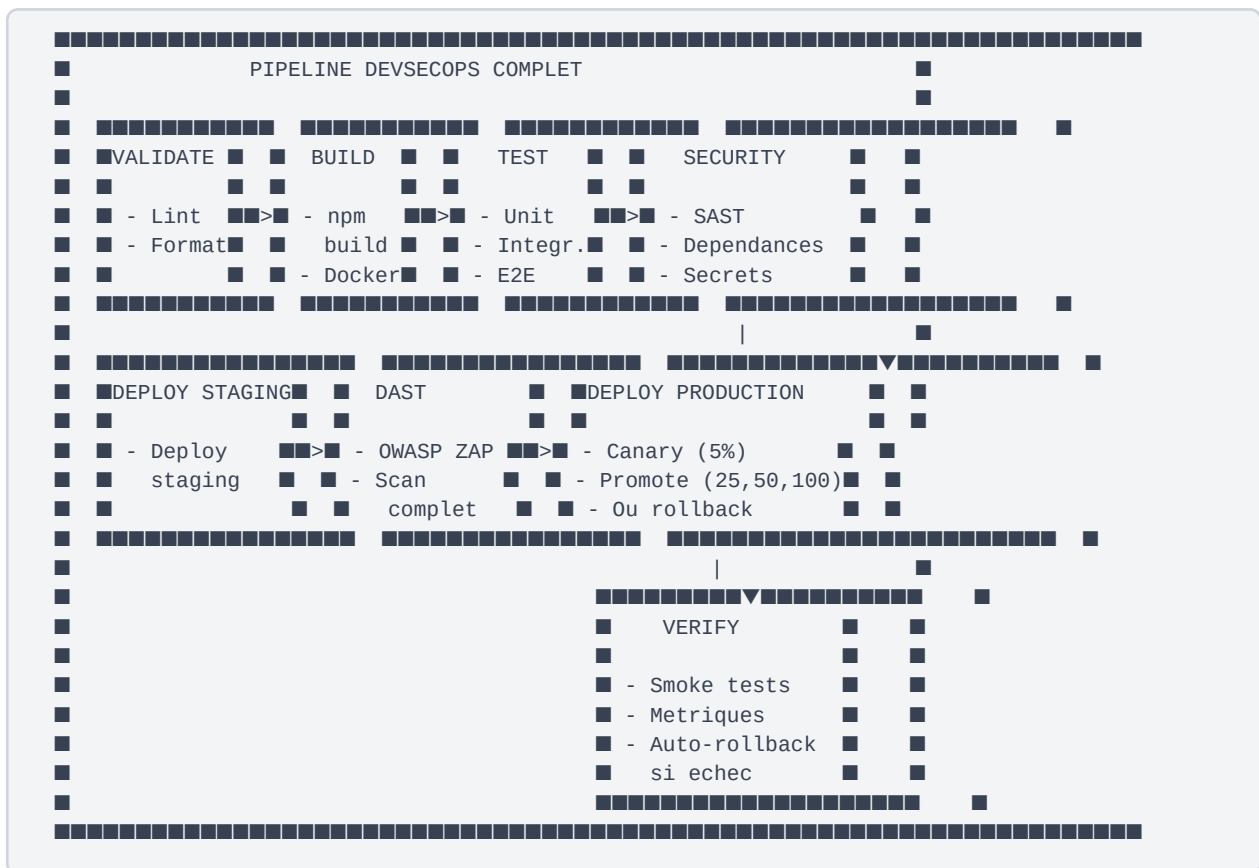
verify-production:
  stage: verify
  script:
    - echo "Verification post-deploiement..."
    - 'curl -f https://www.example.com/health || exit 1'
  needs:
    - deploy-production
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# Si verify echoue, on peut rollback
auto-rollback:
  stage: rollback
  script:
    - echo "ROLLBACK AUTOMATIQUE en cours..."
    - echo "Retour a la version precedente"
    # kubectl rollout undo deployment/mon-app
  when: on_failure
  needs:
    - verify-production
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

****Point cle : `when: on\_failure`**** fait que le job de rollback s'execute UNIQUEMENT si le job `verify-production` a echoue. C'est le filet de securite automatique de votre deploiement.

5.7 Resume : le pipeline DevSecOps complet

Voici a quoi ressemble un pipeline complet integrant TOUT ce que nous avons vu aujourd'hui :



6. Recapitulatif et checklist

Ce que nous avons appris aujourd'hui

Concept	Phrase cle
DevSecOps	La securite fait partie du pipeline, pas un ajout a la fin
Shift Left	Detecter les problemes le plus tot possible = moins cher
SAST	Analyser le code source sans l'executer
DAST	Tester l'application en cours d'execution
Dependances	Verifier que les librairies externes n'ont pas de failles
Secrets	Ne JAMAIS mettre de mots de passe dans le code
GitOps	Git est la seule source de verite pour l'infrastructure
Declaratif	Decrire l'etat desire, pas les etapes pour y arriver
Pull vs Push	Le modele Pull (GitOps) est plus securise
ArgoCD / Flux	Outils GitOps pour Kubernetes
Delivery vs Deploy	Delivery = bouton manuel, Deployment = automatique
Feature Flags	Activer/desactiver une fonctionnalite sans redeployer
Canary	Deployer sur un petit % d'abord, puis augmenter
Rollback	Ctrl+Z du deploiement : revenir a la version precedente
Monitoring	Surveiller metriques, logs et traces apres deploiement

Checklist DevSecOps pour vos projets

- [] Le fichier `.gitignore` exclut les fichiers sensibles (`.env`, `cles`, etc.)
- [] Les secrets sont dans les variables CI/CD de GitLab, PAS dans le code
- [] Le pipeline inclut un scan SAST (Semgrep ou template GitLab)
- [] Le pipeline inclut `npm audit` pour les dependances
- [] Le pipeline inclut la detection de secrets (gitleaks ou template GitLab)
- [] Un deploiement staging existe AVANT la production
- [] Le deploiement production a une strategie de rollback
- [] Des smoke tests verifient l'application apres deploiement
- [] Le monitoring est en place (metriques, logs, alertes)
- [] Toute modification d'infrastructure passe par une Merge Request

Les analogies a retenir

Concept	Analogie
DevSecOps	Expert securite present des la construction de la maison
Shift Left	Mini-contrôles chaque semaine plutot qu'un examen final
SAST	Correcteur orthographique automatique
DAST	Test d'intrusion physique d'un batiment
Dependances	Verifier la date de peremption des ingredients
Secrets	Ne pas coller son code de carte bancaire sur un Post-it
GitOps	Le plan d'architecte : tout est ecrit et versionne
Reconciliation	Le thermostat qui maintient la temperature
Feature Flags	L'interrupteur de lumiere : ON/OFF sans travaux
Canary	Le canari dans la mine : detecter le danger tot
Rollback	Ctrl+Z : annuler la derniere action
Monitoring	Le tableau de bord de la voiture

****Prochain jour**** : nous mettrons en pratique tous ces concepts avec des exercices concrets sur un vrai projet, en configurant un pipeline DevSecOps complet avec GitOps.

Jour 4 - DevSecOps et GitOps - Securite et Deploiement Continu

Formation CI/CD